

1 Трансфомер

1.1 Преобразование одной последовательности в другую

Рассмотрим задачу преобразования одной последовательности в другую (seq2seq). Это может быть, например, машинный перевод или чат-бот, который должен выдавать ответ на вопрос. Какие модели обработки последовательностей нам пока известны?

Это RNN, LSTM и GRU, а также одномерные свёрточные нейронные сети. Стоит также вспомнить архитектуру кодировщик-декодировщик (encoder-decoder). В ней один блок, кодировщик, кодирует исходную последовательность в контекстное представление, затем другой блок, декодировщик, опираясь на него генерирует выходную последовательность токен за токеном, пока не будет выведен токен $\langle \text{EOS} \rangle$.

Такая архитектура позволяет естественно работать с входными и выходными последовательностями разной длины.

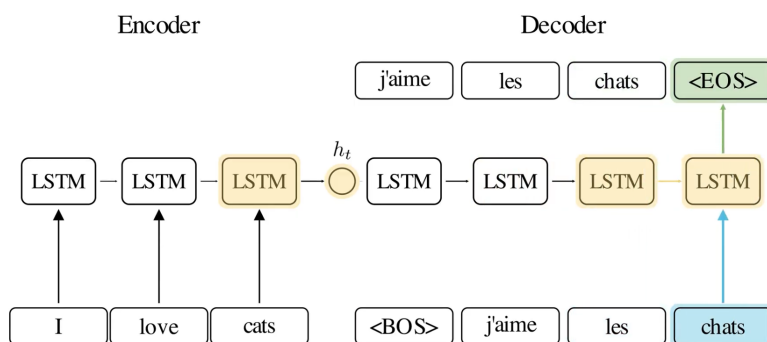


Рис. 1: Архитектура LSTM кодировщик-декодировщик.

Проблема такого подхода в том, что при обработке токенов тяжело учитывать дальние зависимости. Для решения этой проблемы в рекуррентные архитектуры стали добавлять механизмы внимания (attention), позволяющие модели обращаться к различным частям входной последовательности напрямую.

Эти методы стали в итоге state-of-the-art в задачах обработки последовательных данных. Однако они имеют существенные недостатки: подобные архитектуры сложно масштабировать, а их вычисления трудно распараллелить. Из-за своей рекуррентной природы такие сети вынуждены обрабатывать токены последовательно, поскольку за счёт этой архитектурной особенности закладывается индуцированная предвзятость (inductive bias) о порядке токенов в обрабатываемом тексте.

А что если мы откажемся от рекуррентности вообще и сделаем ставку только на механизм внимания? Вдруг окажется так, что внимание — это всё, что нам нужно? В 2017 году была предложена архитектура, которая полностью отказывается от рекуррентности и свёрток и полагается только на механизмы внимания. Эта архитектура называется трансформер.

1.2 Внимание (Attention)

Рассмотрим основу архитектуры трансформеров — внимание.

1.2.1 Мультипликативное внимание (Multiplicative attention)

Что вообще такое внимание? Этим термином будем обозначать силу связи между элементами (векторами) двух последовательностей — $G = \{g_i\}_{i=1}^{s_1} \in \mathbb{R}^{s_1 \times D}$ и $H = \{h_j\}_{j=1}^{s_2} \in \mathbb{R}^{s_2 \times D}$. Сделаем упрощение, что вектора этих последовательностей находятся в общем семантическом пространстве. Тогда для векторов, g_i и h_j , соответствующих хорошо согласованным или близким по смыслу элементам, скалярное произведение $g_i h_j^\top$ будет большим.

На основе скалярного произведения строится механизм мультипликативного внимания:

$$e_{ij} = g_i h_j^\top$$

Чтобы придать большую выразительную силу этому механизму, мы можем использовать билинейную форму:

$$e_{ij} = g_i W h_j^\top,$$

где:

$$W \in \mathbb{R}^{D \times D},$$

или же в матричном виде:

$$E = G W H^\top$$

Так мы можем учитывать, какие компоненты в семантическом пространстве имеют больший вес на матрицу оценок внимания E , а какие — меньший.

Естественно, мы не будем вручную проставлять веса для различных компонент, а сделаем матрицу W обучаемой матрицей весов.

1.2.2 Самовнимание (Scaled-dot Self-Attention)

Вывод самовнимания Связь между векторами g_i и h_j не всегда имеет смысл моделировать одной матрицей W . Выполним небольшой трюк: наложим на W условие возможного разложения на две матрицы такой же размерности, назовём их W_Q и W_K . Тогда:

$$W = W_Q W_K^\top$$

где:

$$W_Q \in \mathbb{R}^{D \times D}, \quad W_K \in \mathbb{R}^{D \times D},$$

Перепишем выражение мультипликативного внимания следующим образом:

$$E = G W H^\top = G (W_Q W_K^\top) H^\top = (G W_Q) (W_K^\top H^\top),$$

поскольку для матриц верно следующее свойство

$$B^\top A^\top = (AB)^\top,$$

то

$$E = (G W_Q) (H W_K)^\top$$

До сих пор мы рассматривали внимание между двумя различными последовательностями векторов G и H . Поскольку наша цель — решить задачу обработки одной последовательности $X = \{x_i\}_{i=1}^s$, то будем вычислять не внимание между двумя разными последовательностями, а самовнимание (self-attention) внутри последовательности X :

$$E = (X W_Q) (X W_K)^\top$$

Пусть $X W_Q = Q$ и $X W_K = K$. Тогда получим матрицу оценок самовнимания последовательности X

$$E = Q K^\top$$

Матрица весов внимания Каждая строчка матрицы E отвечает за один вектор q_i , который смотрит на все векторы $\{k_j\}_{j=1}^s$, и дает оценку, насколько сильно вектор q_i должен обратить внимание на вектор k_j

$$e_{ij} = q_i k_j^\top$$

для фиксированного q_i мы получаем строку оценок

$$e_i = (e_{i1}, e_{i2}, \dots, e_{is})$$

применим к этой строке функцию Softmax, чтобы получить нормированное распределение весов внимания:

$$a_i = \text{Softmax}(e_i)$$

или же в матричной форме:

$$A = \text{Softmax}(QK^\top),$$

где Softmax применяется независимо по строкам.

В такой форме мы избегаем появления отрицательных чисел, все значения нормализованы к интервалу $(0, 1)$, то есть находятся в одном масштабе, и $\sum_j a_{ij} = 1$.

Матрицу весов внимания можно трактовать следующим образом: у каждого вектора q_i есть 'бюджет' внимания, равный 1. Он может построить распределение этого 'бюджета' между всеми векторами $\{k_j\}_{j=1}^s$.

Нормализация матрицы внимания Для двумерного случая, функция $\text{Softmax}(x)$ почти то же самое, что и сигмоида — $\sigma(x)$. Как мы помним, основная проблема $\sigma(x)$ заключается в быстром насыщении. Softmax страдает от той же проблемы.

При вычислении матрицы внимания, нам необходимо найти скалярное произведение

$$q_i k_j^\top = \sum_{k=1}^D q_i^{(k)} k_j^{(k)},$$

отсюда видно, что при большем значении D стоит ожидать больших по модулю значений $q_i k_j^\top$. Это приведет к перенасыщению Softmax при работе с векторами большой размерности. В свою очередь, перенасыщение Softmax крайне негативно скажется на способности модели к масштабированию, поскольку стабильность обучения будет падать при увеличении размерности векторов.

Оценим среднее и дисперсию суммы

$$\sum_{k=1}^D q_i^{(k)} k_j^{(k)}$$

Поскольку

$$\mathbb{E}[q_i^{(k)}] = 0, \quad \text{Var}[q_i^{(k)}] = 1$$

и

$$\mathbb{E}[k_j^{(k)}] = 0, \quad \text{Var}[k_j^{(k)}] = 1,$$

то, сделав предположение о независимости $q_i^{(k)}$ и $k_j^{(k)}$, получаем

$$\mathbb{E}[q_i^{(k)} k_j^{(k)}] = 0, \quad \text{Var}[q_i^{(k)} k_j^{(k)}] = 1,$$

Если дополнительно предположить, что $q_i^{(k)} k_j^{(k)}$ независимы по k , то

$$\mathbb{E}\left[\sum_{k=1}^D q_i^{(k)} k_j^{(k)}\right] = 0, \quad \text{Var}\left[\sum_{k=1}^D q_i^{(k)} k_j^{(k)}\right] = D,$$

Чтобы снизить дисперсию результата скалярного произведения до 1, разделим его на $\sqrt{D}$. Таким образом получаем нормализацию

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{D}}\right),$$

которая предотвращает чрезмерное насыщение Softmax.

Перевзвешивание токенов на основе самовнимания Однако, одних весов внимания A недостаточно. Матрица A показывает лишь, каким образом каждый вектор распределяет свое внимание между остальными векторами последовательности. Однако, сама по себе, она не выполняет никаких преобразований и не говорит, какие вектора будут преобразовываться.

Самый простой вариант — агрегировать исходные вектора последовательности X :

$$O = AX$$

В этом случае, каждый вектор o_i будет являться взвешенной суммой (а поскольку $\sum_j a_{ij} = 1$, то можно говорить о средневзвешенном) исходных векторов x_j :

$$o_i = \sum_{j=1}^s a_{ij} x_j$$

Но такой подход ограничен. Как ранее в мультипликативном внимании мы ввели обучаемую матрицу W (которую потом разделили на две обучаемые матрицы W_Q и W_K), так и сейчас введем преобразование с помощью обучаемой матрицы W_V

$$V = XW_V,$$

чтобы таким образом повысить выразительность итогового преобразования

$$O = AV$$

Можно сказать о том, что матрица W_V является фильтром, определяющим с каким весом и какую информацию извлекать из каждой компоненты вектора x_j :

$$v_j = x_j W_V = \begin{pmatrix} x_{j1} \\ x_{j2} \\ \dots \\ x_{jD} \end{pmatrix}^\top \begin{pmatrix} w_{v11} & w_{v12} & \dots & w_{v1D} \\ \dots & & & \\ w_{vD1} & w_{vD2} & \dots & w_{vDD} \end{pmatrix} = \begin{pmatrix} \sum_{k=1}^D x_{jk} w_{vk1} \\ \sum_{k=1}^D x_{jk} w_{vk2} \\ \dots \\ \sum_{k=1}^D x_{jk} w_{vkD} \end{pmatrix}^\top$$

Переставим местами x_{jk} и w_{vki} и увидим, что столбцы матрицы W_V представляют собой веса взвешенной суммы по компонентам вектора x_j :

$$v_j = \begin{pmatrix} \sum_{k=1}^D x_{jk} w_{v_{k1}} \\ \sum_{k=1}^D x_{jk} w_{v_{k2}} \\ \dots \\ \sum_{k=1}^D x_{jk} w_{v_{kD}} \end{pmatrix}^T = \begin{pmatrix} \sum_{k=1}^D w_{v_{k1}} x_{jk} \\ \sum_{k=1}^D w_{v_{k2}} x_{jk} \\ \dots \\ \sum_{k=1}^D w_{v_{kD}} x_{jk} \end{pmatrix}^T$$

Таким образом, общая форма для scaled-dot self-attention выглядит следующим образом:

$$O = \text{Softmax}\left(\frac{QK^T}{\sqrt{D}}\right)V$$

Матрицы Q , K и V называются соответственно матрицами **запросов** (**queries**), **ключей** (**keys**) и **значений** (**values**).

Что такое запросы, ключи и значения Элемент матрицы оценок внимания $e_{ij} = q_i k_j^T$ представляет собой силу связи вектора запроса q_i и вектора ключа k_j . А строка $e_i = q_i K^T$ представляет собой набор оценок того, насколько сильно вектор q_i должен обратить внимание на каждый из векторов ключей последовательности.

Отсюда вектор q_i можно трактовать как **запрос**, какую именно информацию текущий элемент последовательности хочет найти в остальных элементах. А вектор k_j можно трактовать как **ключ**, по которому другие элементы последовательности могут понять, насколько данный элемент подходит их запросу.

Если запрос q_i и ключ k_j близки, то скалярное произведение получается высоким, соответственно внимание от элемента x_i к элементу x_j повышается.

Как уже было отмечено, столбцы матрицы W_V представляют собой веса взвешенной суммы по компонентам вектора x_j . Соответственно вектор $v_j = x_j W_V$ представляет собой **значения**, которые, согласно матрице W_V , должны передаваться дальше.

1.2.3 Многоглавое внимание (Multihead Attention)

Механизм самовнимания, построенный на одной тройке матриц $\{W_Q, W_K, W_V\}$ позволяет построить только одно распределение внимания. Однако, в реальных данных зависимости между элементами могут быть совершенно разными, и их может быть трудно описать с помощью одной тройки пространств ключей, запросов и значений.

Поэтому, вместо одного механизма самовнимания, будем использовать несколько параллельных и независимых механизмов — голов внимания. Так, каждая голова будет описываться своими собственными матрицами $\{W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}\}$, а значит может искать зависимости в своих собственных пространствах.

Пусть число голов внимания равно H . Для каждой головы h

$$W_Q^{(h)} \in \mathbb{R}^{D \times d_h} \quad W_K^{(h)} \in \mathbb{R}^{D \times d_h} \quad W_V^{(h)} \in \mathbb{R}^{D \times d_h}$$

где $d_h = \frac{D}{H}$, чтобы после объединения голов сохранить исходную размерность.

Запишем результат механизма самовнимания для головы h :

$$O^{(h)} = \text{Softmax}\left(\frac{Q^{(h)} K^{(h)\top}}{\sqrt{d_h}}\right) V^{(h)}$$

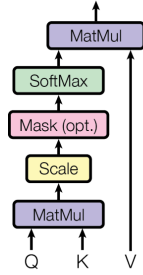
Объединим головы:

$$O_{\text{concat}} = \text{Concat}[O^{(1)}, O^{(2)}, \dots, O^{(H)}]$$

Выходы разных голов находятся в разных пространствах. Пусть, после объединения голов мы и получили матрицу O_{concat} той же размерности, что и исходная матрица X , информация, полученная из разных голов, все еще разделена покомпонентно. Чтобы перемешать информацию из разных голов, добавим еще одно обучаемое линейное преобразование $W_O \in \mathbb{R}^{D \times D}$:

$$O = O_{\text{concat}} W_O$$

Scaled Dot-Product Attention



Multi-Head Attention

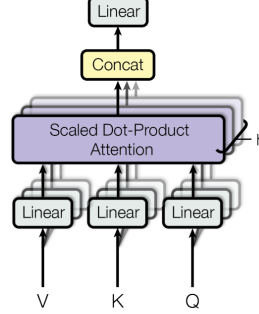


Рис. 2: Сравнение механизма самовнимания и многоглавого внимания. Взято из оригинальной статьи “Attention is all you need”.

На первый взгляд может показаться, что механизм многоглавого внимания требует значительно больше вычислений и параметров, чем механизм самовнимания с одной головой. На деле, увеличение сложности лишь связано с появлением нового слоя W_O .

1.2.4 Маскированное многоглавое внимание (Masked Multihead Attention)

До сих пор мы рассматривали лишь случай, когда каждый элемент последовательности всегда может обратить внимание на любой другой элемент. Однако, это не всегда допустимо, в некоторых задачах необходимо заранее ограничить множество токенов, на которые разрешено смотреть. Для этого в механизм внимания вводится маска.

Мы могли бы обнулять некоторые элементы a_{ij} матрицы весов внимания. Но тогда мы ломаем правило, что сумма элементов каждой строки a_i равна 1, а также сместим распределение весов внимания. Поэтому, чтобы не приходилось снова нормализовать значения весов, удобнее накладывать маску до вычисления Softmax.

Вспомним формулу Softmax:

$$\text{Softmax}_j(x) = \frac{\exp(x^{(j)})}{\sum_{k=1}^D \exp(x^{(k)})}$$

$\text{Softmax}_j(x) = 0$ тогда, когда $\exp(x^{(j)}) = 0$. А $\exp(x^{(j)}) = 0$ тогда, когда аргумент экспоненты равен $-\infty$.

Таким образом, чтобы наложить маску на матрицу весов A , необходимо к матрице оценок внимания E прибавить матрицу M , которая состоит из $-\infty$ на тех местах, которые необходимо маскировать, и из нулей во всех остальных случаях.

1.2.5 Перекрестное внимание (Cross Attention)

Ранее, при переходе от мультипликативного внимания к самовниманию, мы сделали упрощение, ограничив вход только одной последовательностью X .

$$E = (GW_Q)(W_K^T H^T) = \tilde{Q}\tilde{K}^T$$

Но, все последующие выкладки никак не противоречат тому, что последовательности $\tilde{Q}$ и $\tilde{K}$ могут происходить из разных источников и быть разной длины — $s_{\tilde{Q}}$ и $s_{\tilde{K}}$. Для корректности вычислений необходимо только, чтобы

$$s_{\tilde{V}} = s_{\tilde{K}},$$

то есть длина последовательности $\tilde{V}$ была равна длине последовательности $\tilde{K}$.

Пусть матрица $\tilde{Q}$ происходит из последовательности входов X

$$\tilde{Q} = XW_Q,$$

а матрицы $\tilde{K}$ и $\tilde{V}$ происходят из другой последовательности — Z :

$$\tilde{K} = ZW_K, \quad \tilde{V} = ZW_V,$$

Механизм внимания, построенный на основе $\{\tilde{Q}, \tilde{K}, \tilde{V}\}$ называется **перекрестным вниманием (cross-attention)**.

$$O = \text{Softmax}\left(\frac{\tilde{Q}\tilde{K}^\top}{\sqrt{d_h}}\right)\tilde{V}$$

Перекрестное внимание позволяет вычислять внимание между разными последовательностями.

1.3 FFN-слой

Механизм внимания отвечает за обмен информацией между различными токенами входной последовательности. Но после этого еще необходимо обработать полученное представление каждого токена более сложным и нелинейным образом.

Для этого используется двуслойная нейронная сеть, называемая FFN-слоем. FFN в трансформере применяется независимо к каждому токenu входной последовательности X .

$$\text{FFN}(x_i) = \phi(x_i W_1 + b_1) W_2 + b_2,$$

где

$$W_1 \in \mathbb{R}^{D \times D_{FFN}}, \quad W_2 \in \mathbb{R}^{D_{FFN} \times D}$$

Обычно, размерность внутреннего слоя D_{FFN} выбирается больше, чем D . FFN сначала расширяет пространство признаков, а следом сжимает.

При обучении трансформера в FFN-слое используют dropout. Его обычно применяют между двумя полносвязными слоями.

$$\text{FFN}(x_i) = \text{Dropout}\left[\phi(x_i W_1 + b_1)\right] W_2 + b_2,$$

Иногда dropout применяют и после второго линейного преобразования. FFN-слой содержит большое количество параметров и выполняется независимо для каждого токена. Из-за этого возможно переобучение, которое и компенсируется регуляризацией.

1.4 Нормализация

LayerNorm и BatchNorm основаны на одной идее нормализации активаций. Разница заключается в том, как мы считаем $\mathbb{E}[X]$ и $\text{Var}[X]$. BatchNorm вычисляет эти статистики по батчу, LayerNorm же вычисляет статистики по одному объекту.

LayerNorm работает одинаково при обучении и инференсе, в отличие от BatchNorm, который требует агрегирования статистик по батчу во время обучения и использует накопленные статистики для инференса. Также,

LayerNorm, в отличие от BatchNorm, лучше применять к последовательностям разной длины и к малым батчам, так как статистики BatchNorm в этих случаях становятся более шумными.

Кроме того, LayerNorm лучше подходит для малых батчей, так как статистики BatchNorm при этом становятся слишком шумными.

1.5 Слой трансформера

Соберем все модули вместе: механизм многоглавого внимания (МНА), LayerNorm (LN) и FFN-слой. Попробуем применить их последовательно:

$$\text{TransformerLayer}(x) = \text{LN} \circ \text{FFN} \circ \text{LN} \circ \text{МНА}(x)$$

Наивный подход, когда мы применяем преобразования одно за другим, оказывается не слишком удачным для обучения. Даже один такой слой уже содержит в себе четыре последовательных матричных умножения из механизма многоглавого внимания, функцию Softmax, две нормализации, два линейных преобразования из FFN-слоя и функцию активации.

1.5.1 Остаточные связи (Residual connections)

Уже в внутри одного трансформерного слоя мы рискуем столкнуться с проблемой затухающих градиентов. А в реальной модели потребуется несколько таких слоёв. Чтобы этого избежать, к выходу каждого крупного подслоя прибавляется его вход — остаточная связь (residual connection):

$$z = \text{LN}\left(x + \text{МНА}(x)\right),$$

$$y = \text{LN}\left(z + \text{FFN}(z)\right),$$

Попробуем продифференцировать такие преобразования по входу (будем рассматривать частную производную компоненты выхода y по компоненте входа x), для второго подслоя тогда получим

$$\frac{\partial y}{\partial z} = \frac{\partial \text{LN}\left(z + \text{FFN}(z)\right)}{\partial \left(z + \text{FFN}(z)\right)} \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right),$$

а для первого

$$\frac{\partial z}{\partial x} = \frac{\partial \text{LN}\left(x + \text{МНА}(x)\right)}{\partial \left(x + \text{МНА}(x)\right)} \left(1 + \frac{\partial \text{МНА}(x)}{\partial x}\right)$$

Пусть $\tilde{z} = z + \text{FFN}(z)$ и $\tilde{x} = x + \text{МНА}(x)$. Найдём $\frac{\partial y}{\partial x}$:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial x} = \frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right) \cdot \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})} \left(1 + \frac{\partial \text{MHA}(x)}{\partial x}\right),$$

сгруппируем производные LayerNorm по входу, для этого рассмотрим частную производную:

$$\frac{\partial y}{\partial x} = \frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \cdot \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})} \cdot \left(1 + \frac{\partial \text{FFN}(z)}{\partial z}\right) \cdot \left(1 + \frac{\partial \text{MHA}(x)}{\partial x}\right),$$

раскроем скобки:

$$\frac{\partial y}{\partial x} = \underbrace{\frac{\partial \text{LN}(\tilde{z})}{\partial \tilde{z}} \frac{\partial \text{LN}(\tilde{x})}{\partial(\tilde{x})}}_{\text{Градиенты почти не затухают}} \cdot \underbrace{\left(1 + \frac{\partial \text{FFN}(z)}{\partial z} + \frac{\partial \text{MHA}(x)}{\partial x}\right)}_{\text{Градиенты затухают слабее}} + \underbrace{\frac{\partial \text{FFN}(z)}{\partial z} \cdot \frac{\partial \text{MHA}(x)}{\partial x}}_{\text{Градиенты сильно затухают}}$$

Таким образом видно, что добавление остаточных связей позволяет градиентам доходить до более глубоких слоев, почти избегая затухания.

1.6 Позиционное кодирование

Рассмотрим слой трансформера. Сначала идет механизм внимания. Механизм внимания смотрит на все элементы последовательности разом, преобразование вектора никак не зависит от его порядка, а зависит от значений других векторов последовательности. Далее идет слой LayerNorm, который обрабатывает каждый token независимо от других. Следом — FFN-слой, который также обрабатывает каждый token независимо.

В рекуррентных сетях мы закладывали индуктивное предубеждение о порядке токенов за счёт порядка обработки элементов последовательности. В трансформере же мы обрабатываем все элементы разом. Мы нигде не добавляем информацию о порядке слов в предложении.

Между тем, порядок слов в предложении крайне важен, поскольку передает логический акцент и определяет контекст. Например, в немецком языке крайне важно располагать частицу *nicht* в нужном месте, чтобы не исказить смысл:

- Er kommt morgen **nicht** — Он не придет завтра,
- Er kommt **nicht** morgen — Он придет, но не завтра.

В этих предложениях используются абсолютно одинаковые слова, но смысл изменяется на противоположный, когда мы поменяли расположение **nicht**.

Поскольку пока мы никак не добавляем в модель информацию о позиции токенов, добавим эту информацию к самой последовательности.

1.6.1 Абсолютное позиционное кодирование

К каждому токenu прибавим некоторый вектор, который будет кодировать его абсолютную позицию в последовательности:

$$\hat{x}_{\text{pos}} = x_{\text{pos}} + \text{PE}(\text{pos}),$$

где $\text{PE} : \mathbb{N} \rightarrow \mathbb{R}^D$ — функция, отображающая позицию токена (число) в вектор позиционного кодирования, размерность которого совпадает с размерностью вектора обрабатываемой последовательности.

Для такой функции обязательно должно выполняться то, что разные позиции должны кодироваться разными векторами. Также желательно, чтобы позиции имели близкие вектора и кодирование было таким, чтобы модель была способна восстановить относительные сдвиги между позициями.

1.6.2 Синусоидальное кодирование

Последнее особенно важно, поскольку было бы удобно, если бы кодировка позиции $\text{pos} + k$ могла быть выражена через кодировку позиции pos и величину сдвига k . Для этого естественно использовать синус и косинус, поскольку для этих функций выполняется:

$$\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta,$$

$$\cos(\alpha + \beta) = \cos \alpha \cos \beta - \sin \alpha \sin \beta.$$

Если теперь в некоторой компоненте вектора позиционного кодирования записать $\sin(\omega \text{pos})$, а в соседней — $\cos(\omega \text{pos})$, то для позиции $\text{pos} + k$ получим:

$$\sin(\omega(\text{pos} + k)) = \sin(\omega \text{pos}) \cos(\omega k) + \cos(\omega \text{pos}) \sin(\omega k),$$

$$\cos(\omega(\text{pos} + k)) = \cos(\omega \text{pos}) \cos(\omega k) - \sin(\omega \text{pos}) \sin(\omega k).$$

таким образом, кодировка позиции $\text{pos} + k$ в паре компонент с одной и той же частотой ω выражается через кодировку позиции pos .

Выберем набор частот, поскольку если использовать одну частоту, то разные позиции начнут периодически совпадать. Покомпонентно используются сразу многие частоты, это изменять одни компоненты быстро, а другие медленно, кодируя тем самым как локальное положение токена, так и глобальное:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/D}}\right),$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/D}}\right),$$

где i — номер пары компонент. Малые i соответствуют низким частотам, большие i — высоким. Итоговый вектор выглядит следующим образом:

$$PE_{pos} = \begin{pmatrix} \sin\left(\frac{pos}{10000^{0/D}}\right) \\ \cos\left(\frac{pos}{10000^{0/D}}\right) \\ \sin\left(\frac{pos}{10000^{2/D}}\right) \\ \cos\left(\frac{pos}{10000^{2/D}}\right) \\ \vdots \\ \cos\left(\frac{pos}{10000^{(D-2)/D}}\right) \\ \sin\left(\frac{pos}{10000^{(D-2)/D}}\right) \end{pmatrix}^T$$

Число 10000 задает диапазон используемых частот. У каждой пары компонент свой период:

$$T_i = \frac{2\pi}{w_i} = 2\pi \cdot 10000^{2i/D},$$

так что использование различных частот для каждой компоненты также позволяет выполнить требование неравенства векторов позиционного кодирования для разных позиций.

1.7 Трансформер кодировщик и трансформер декодировщик

Рассмотрим архитектуру трансформера-кодировщика. Его задача — некоторым образом кодировать входную последовательность X_1 в последовательность Z . Архитектура представлена рядом из N_{enc} последовательных слоёв.

Каждый такой слой можно записать следующим образом:

$$\tilde{x} = \text{LN}\left(x + \text{MHA}(x)\right),$$

$$y = \text{LN}\left(\hat{x} + \text{FFN}(\hat{x})\right),$$

Теперь рассмотрим архитектуру трансформера-декодировщика. В отличие от кодировщика, декодировщик преобразует последовательность X_2 не в какое то латентное представление Z , а в другую последовательность Y , которая является продолжением последовательности X_2 . Трансформер-декодировщик предсказывает новую последовательность токенов за токеном, но об обучении трансформера немного позже.

Важно, что обычный механизм внимания не подходит для этой задачи, поскольку тогда каждый токен будет видеть все следующие, что позволяет модели переобучаться, как бы заглядывая в будущее. Поэтому будем маскировать внимание, прибавив к матрице оценок внимания маску.

В этой маске на всех позициях выше главной диагонали будет значение $-\infty$, а на главной диагонали и ниже — нули. Это позволить обнулить веса внимания для всех следующих токенов последовательности:

$$\text{MaskedMHA}(x) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V,$$

где:

$$m_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i \end{cases}$$

Запишем теперь слой трансформера-декодировщика:

$$\tilde{x} = \text{LN}\left(x + \text{MaskedMHA}(x)\right),$$

$$y = \text{LN}\left(\tilde{x} + \text{FFN}(\tilde{x})\right),$$

Также, как и кодировщик, трансформер-декодировщик можно представить рядом N_{dec} последовательных слоёв.

Теперь рассмотрим архитектуру трансформера кодировщика-декодировщика. Это более общий случай, когда необходимо преобразовать последовательность X_1 в Y . Такая задача ставится, например, при выполнении машинного перевода.

В этом случае, каждый модуль берет на себя свою задачу: кодировщик обрабатывает входную последовательность, декодировщик генерирует выходную последовательность, на основе информации из кодировщика.

Кодировщик видит входную последовательность целиком, поэтому ему достаточно обработать ее один раз. Декодировщик же будет работать авторегрессионно, последовательно генерируя токен за токеном. В этом случае одного только маскированного многоглавого внимания не хватит, поскольку декодировщику, помимо входа X_2 необходимо также учитывать и выход кодировщика Z .

Для этого в слой декодировщика добавляется перекрестное внимание:

$$\tilde{x} = \text{LN}\left(x + \text{MaskedMHA}(x)\right),$$

$$\hat{x} = \text{LN}\left(\tilde{x} + \text{CrossAttention}(\tilde{x}, Z)\right),$$

$$y = \text{LN}\left(\hat{x} + \text{FFN}(\hat{x})\right),$$

В перекрестном внимании последовательность Q происходит из входной последовательности X_2 , а последовательности K и V — из Z .

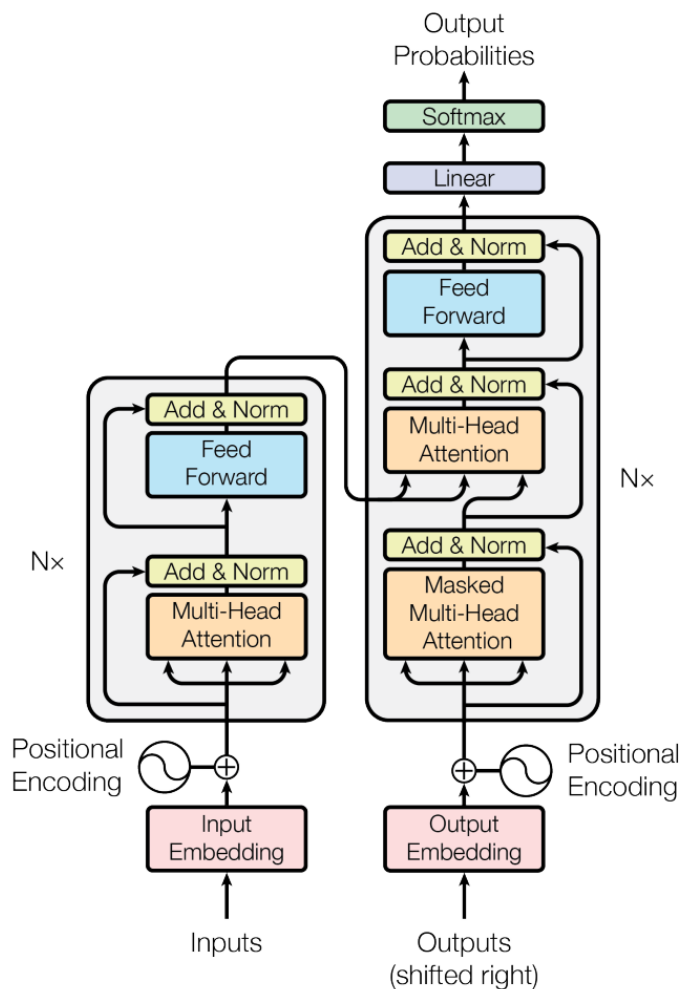


Рис. 3: Архитектура трансформера кодировщика-декодировщика, взято из оригинальной статьи “Attention is all you need”.

1.8 Обучение трансформера

Разберем обучение трансформера на примере декодировщика. Во время обучения необходимо предсказать последовательность токенов, сдвинутую на 1 шаг влево. То есть если на вход идет последовательность

<BOS>, <Архитек> <тура\w>, < >, <Трансфор>, <мер/\w>

то целевая последовательность на выходе выглядит так:

<Архитек> <тура>, < >, <Трансфор>, <мер/w>, <EOS>

Модель должна для каждой позиции входной последовательности предсказать один правильный токен из словаря. Говоря иначе, задача обучения сводится к многоклассовой классификации. Мы должны по каждому из выходных векторов предсказать следующий токен.

Для этого к выходам последнего слоя трансформера применим классификационную голову, состоящую из линейного слоя, преобразующего скрытое состояние токена в вектор логитов размерности словаря. Чтобы получить распределение вероятностей выходного токена q используем Softmax.

Далее, чтобы приблизить это распределение к целевому p , будем минимизировать кросс-энтропию $H(p, q)$ между ними.

Кросс-энтропия выступает мерой неопределенности модели в выборе следующего токена. Вспомним, что мы можем разложить кросс-энтропию на две составляющие части:

$$H(p, q) = H(p) + D_{KL}\langle p||q \rangle,$$

Пусть $\mathcal{C}$ — множество всех возможных контекстов, C — случайная величина, принимающая значения в $\mathcal{C}$, c — конкретный контекст из $\mathcal{C}$, а Y — случайная величина, соответствующая следующему токenu. Для конкретной пары (c, y) , где y — конкретный токен из Y , распределение $p(y|C = c)$ будет вырожденным. Тогда $H(p) = 0$.

Однако, если рассматривать (C, Y) как случайную пару, семплированную из истинного распределения данных, то распределение $p(y|C)$ будет отлично от вырожденного, а значит $H(p) \neq 0$.

$H(p)$ в этом случае мы можем трактовать как энтропию текста, на котором мы обучаем модель. Энтропия языка характеризует неопределенность и непредсказуемость появления следующего токена. Если бы энтропия была равна нулю, то каждый текст был бы предопределен с первого токена. В некотором смысле это шум в данных, который мы не можем устранить. И очевидно, что при обучении значение функции потерь не опустится ниже $H(p)$

$D_{KL}\langle p||q \rangle$ — это та часть, на которую мы в силах повлиять. В естественном языке существует множество правил, которые так или иначе накладывают ограничения на генерацию следующего токена. Это, например, правила грамматики, устойчивые словосочетания, синтаксические зависимости и так далее. В начале обучения модель ничего о них не знает и должна выучить их неявным образом, минимизируя $D_{KL}\langle p||q \rangle$ составляющую кросс-энтропии.

1.8.1 Perplexity

Вычисление кросс-энтропии можно представить как взвешенную сумму логарифмов вероятностей. Такое выражение тяжело интерпретировать. Переделаем его. Возьмем экспоненту от кросс-энтропии:

$$\exp(H(p, q)) = \exp\left(-\sum_i p_i \log q_i\right) = \prod_i \exp(-p_i \log q_i) = \prod_i \exp(\log q_i)^{-p_i}$$

Упростим:

$$\exp(H(p, q)) = \prod_i q_i^{-p_i}$$

Чем выше вероятность, которую модель назначает правильным токенам, тем меньше это значение. И наоборот, если модель распределяет вероятность между большим числом возможных токенов и не уверена в выборе, то значение $\exp(H(p, q))$ растёт.

Эту величину называют перплексией (perplexity):

$$\text{Perplexity}(p, q) = \exp(H(p, q)) = \prod_i q_i^{-p_i}$$

Эту величину можно трактовать как "среднее" число вариантов, между которыми колеблется модель при выборе следующего токена. Например, $\text{Perplexity} = 10$ можно интерпретировать так, будто бы на каждом шаге модель выбирает между 10 равнозначными вариантами.

Перплексию можно использовать в качестве метрики обучения языковой модели и трансформера в частности.

1.9 Инференс трансформера

Во время обучения мы подавали на вход декодировщику текстовую последовательность и учили модель предсказывать новую последовательность, сдвинутую на один шаг влево. Однако во время инференса полной текстовой последовательности ещё нет, модель должна построить её самостоятельно, токен за токеном, начиная с токена $\langle \text{BOS} \rangle$ и генерируя новые токены до тех пор, пока либо не будет достигнут токен $\langle \text{EOS} \rangle$, либо максимальная длина последовательности.

Во время обучения предсказания для всех позиций можно было считать параллельно. Во время инференса мы вынуждены все токены предсказывать последовательно, так как генерация следующего токена зависит от того, какие токены мы сгенерировали на предыдущих шагах.

На одном шаге генерации мы получаем вектор вероятностей. После получения такого вектора, необходимо выбрать токен. Самый простой вариант — использовать жадный алгоритм, всегда выбирая токен с большей вероятностью. Этот подход полностью детерминированный, что может приводить к однообразным ответам и циклам в генерации.

Другой вариант — семплировать токен из полученного распределения. Однако, в этом случае есть шанс генерации маловероятного токена, который никак не подходит под входную последовательность. Компромиссный вариант — семплировать токен из k лучших вариантов с сохранением общего распределения.

1.9.1 Температура трансформера

Для получения вероятностей слов из вектора размера `vocab_size` необходимо применить функцию `Softmax`.

При использовании `Softmax` наибольшему логиту соответствует наибольшая вероятность. Если необходимо регулировать разброс значений вероятностей, можно использовать функцию `Softmax` с температурой:

$$\text{Softmax}_T(x) = \frac{e^{x_k/T}}{\sum_{i=0}^{n-1} e^{x_i/T}}$$

Чем больше температура — тем меньший приоритет отдается большим компонентам. Это добавляет больше случайности при генерации слов в последовательности.

1.10 Масштабируемость трансформеров

Одно из главных преимуществ трансформера перед рекуррентными архитектурами заключается в том, что его вычисления хорошо параллелизируются. Это позволяет значительно ускорить обучение, используя большое количество графических процессоров (GPU).

Большая часть операций внутри трансформера — это матричные умножения (GEMM, General Matrix Multiplication). А GPU хорошо оптимизированы для вычисления матричных умножений.

Итого, параллелизм в трансформере возникает сразу на нескольких уровнях:

1. Параллелизм по батчам
2. Параллелизм по головам
3. Параллельное вычисление GEMM в механизме внимания внутри одной головы

Если раньше масштабирование количества вычислений упиралось в архитектуру модели, то теперь это чисто инженерная задача, заключающаяся в организации вычисления модели на одной видеокарте и на нескольких.